

INF01151 – Sistemas Operacionais II

Aula 20 - Relógios lógicos & Algoritmos de eleição

Prof. Alberto Egon Schaeffer Filho

Introdução

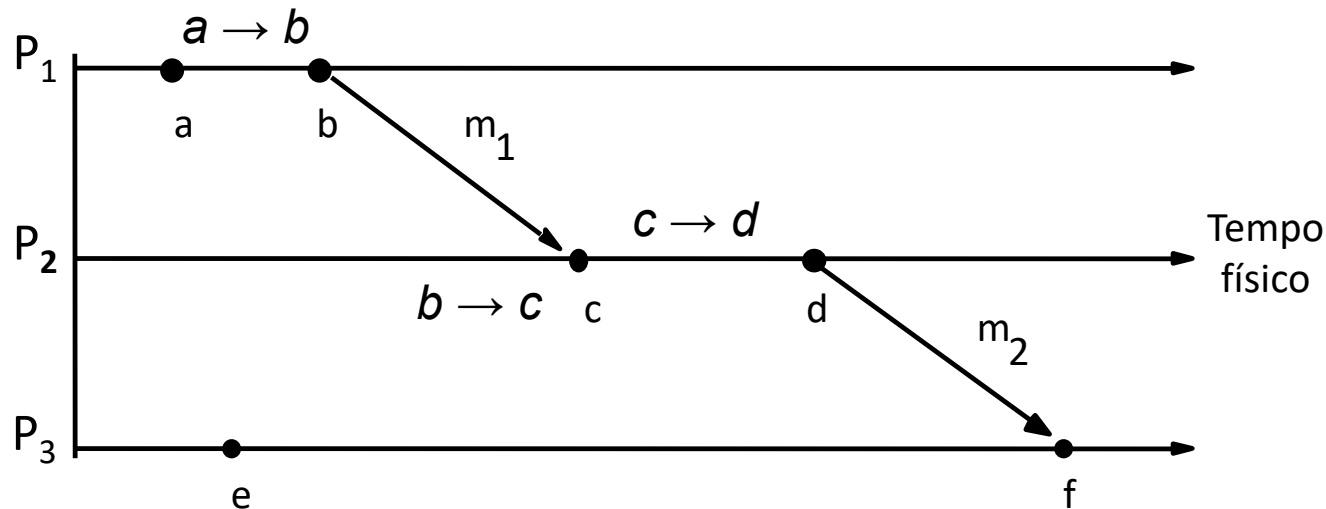
- Como visto na aula passada...
 - Em um único processador, eventos são ordenados pelo relógio local
 - Impossível sincronizar perfeitamente relógios em um sistema distribuído
 - Nem sempre se pode usar o tempo físico para ordenar pares de eventos!
- Solução é utilizar abordagem (intuitiva) de causa-consequência
 - Se dois eventos acontecerem no mesmo processo P_i , então eles ocorreram na ordem que o processo as observou ($\rightarrow_i$)
 - Se uma mensagem é enviada entre processos, o evento de envio da mensagem ocorreu antes do evento de recepção da mensagem
- Ordenação parcial de eventos pode ser obtida através da generalização desses dois pontos acima
 - Relação *happened-before* (by Leslie Lamport)

Notação e definições

- Notação
 - $e_1 \rightarrow_i e_2$
 - Indica que o evento e_1 precede o evento e_2 , no processo P_i
 - $e_1 \rightarrow e_2$
 - Indica que o evento e_1 precede o evento e_2
- Ordenação de eventos
 - Organizar os eventos em uma sequência de tal forma que um preceda o outro
 - Ordem *total*: todos os eventos do sistema estão ordenados
 - Ordem *parcial*: alguns eventos do sistema estão ordenados

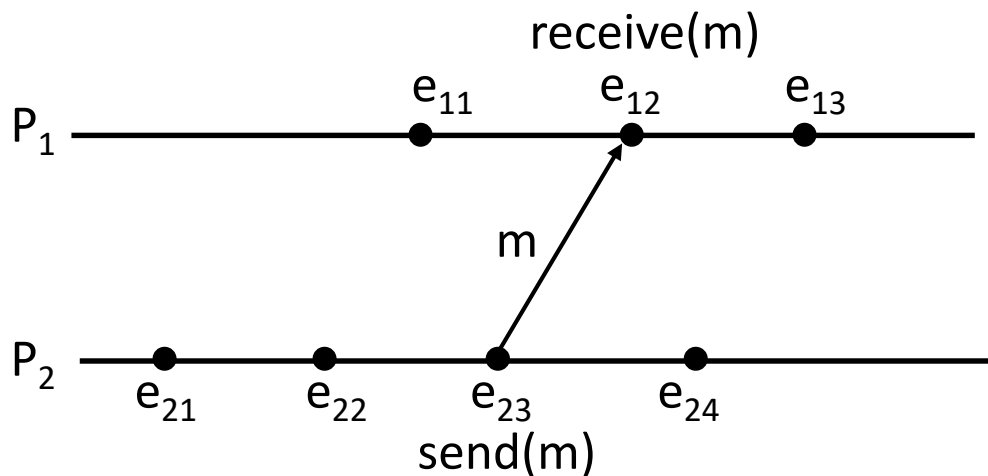
Relação *happened-before*

- Definição da relação *happened-before* ($\rightarrow$)
 - HB1**: se $\exists$ processo P_i : $e \rightarrow_i e'$, então $e \rightarrow e'$
 - HB2**: para qualquer mensagem m , $send(m) \rightarrow receive(m)$
 - HB3**: se e, e' e e'' são eventos tais que $e \rightarrow e'$ e $e' \rightarrow e''$, então $e \rightarrow e''$



Eventos concorrentes: $a \parallel e$ ou seja, não há relação $\rightarrow$ entre eles

Relação *happened-before*



$e_{23} \checkmark \rightarrow e_{12}$ $e_{22} \checkmark \rightarrow e_{12}$ $e_{21} \checkmark \rightarrow e_{12}$ $e_{22} \checkmark \rightarrow e_{13}$ $e_{21} \checkmark \rightarrow e_{13}$

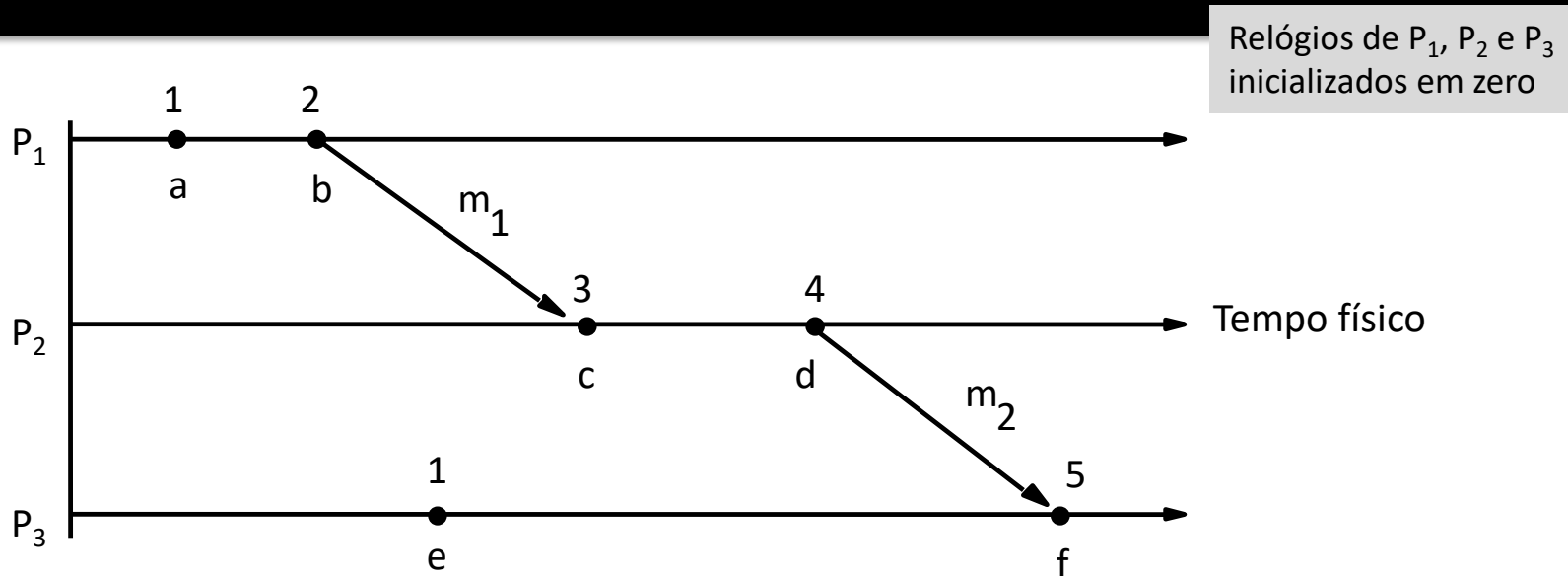
$e_{11} \times \rightarrow e_{21}$ $e_{11} \times \rightarrow e_{22}$ $e_{11} \times \rightarrow e_{23}$ $e_{11} \times \rightarrow e_{24}$

e_{11} é concorrente com e_{21} , e_{22} , e_{23} , e_{24}

Relógio lógico de Lamport

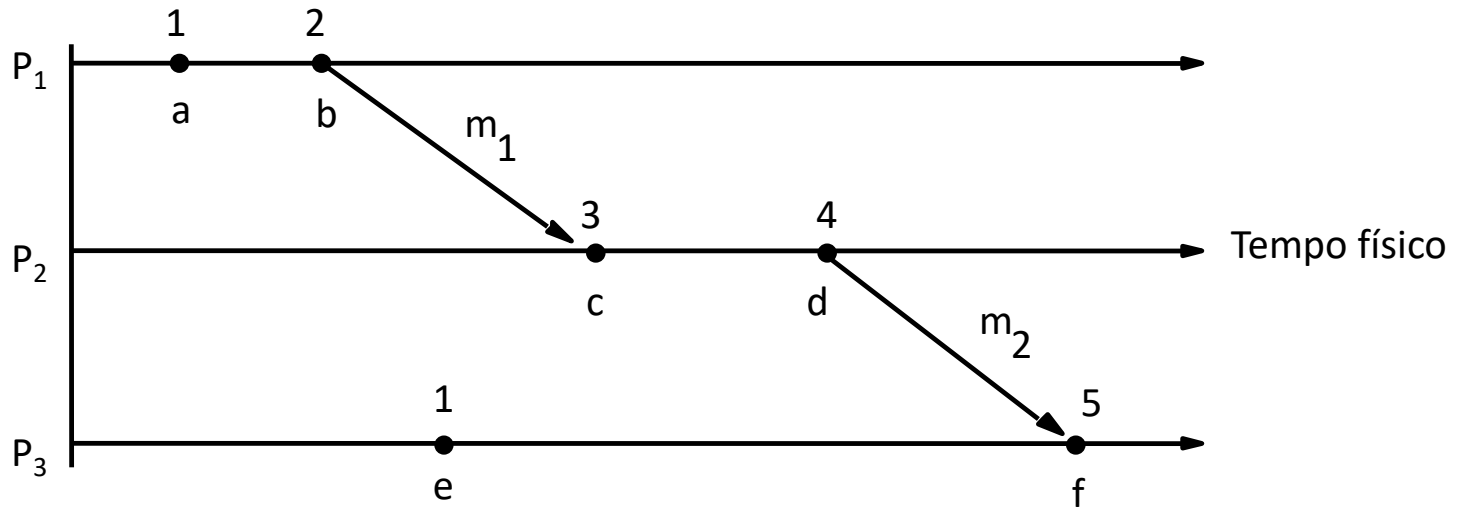
- Forma de “capturar” numericamente relação *happened-before*
 - Contador em software que aumenta a contagem monotonicamente
 - Valor do contador não tem qualquer relação com o relógio físico
- Condições:
 - L_i é o relógio lógico no processo P_i
 - Se $a \rightarrow b$ no processo P_i , $L_i(a) < L_i(b)$
 - Se a é o envio da mensagem m em P_i e b é a recepção da mensagem m em P_j ; então $L_i(a) < L_j(b)$
- Regras para atualização de relógios:
 - **LC1:** L_i é incrementado antes de cada evento ser processado em P_i : $L_i := L_i + 1$
 - **LC2:** **(a)** quando P_i envia mensagem m , ele envia junto valor $t = L_i$
(b) quando P_j recebe (m, t) , ele calcula $L_j := \max(L_j, t)$ e então aplica LC1 para definir o timestamp de $\text{receive}(m)$

Relógio lógico de Lamport: exemplo



- Como obter ordenação total? Eventos distintos em diferentes processos podem ter a mesma indicação de tempo lógico
- Solução é criar uma ordem global (usando algum critério)
 - Incluir identificadores dos processos onde os eventos aconteceram
 - $(T_i, i) < (T_j, j)$, se e somente se $(T_i < T_j)$ ou $(T_i = T_j \text{ e } i < j)$
 - Sem relação temporal real, mas fornece um critério de ordem...

Relógio lógico de Lamport: deficiência

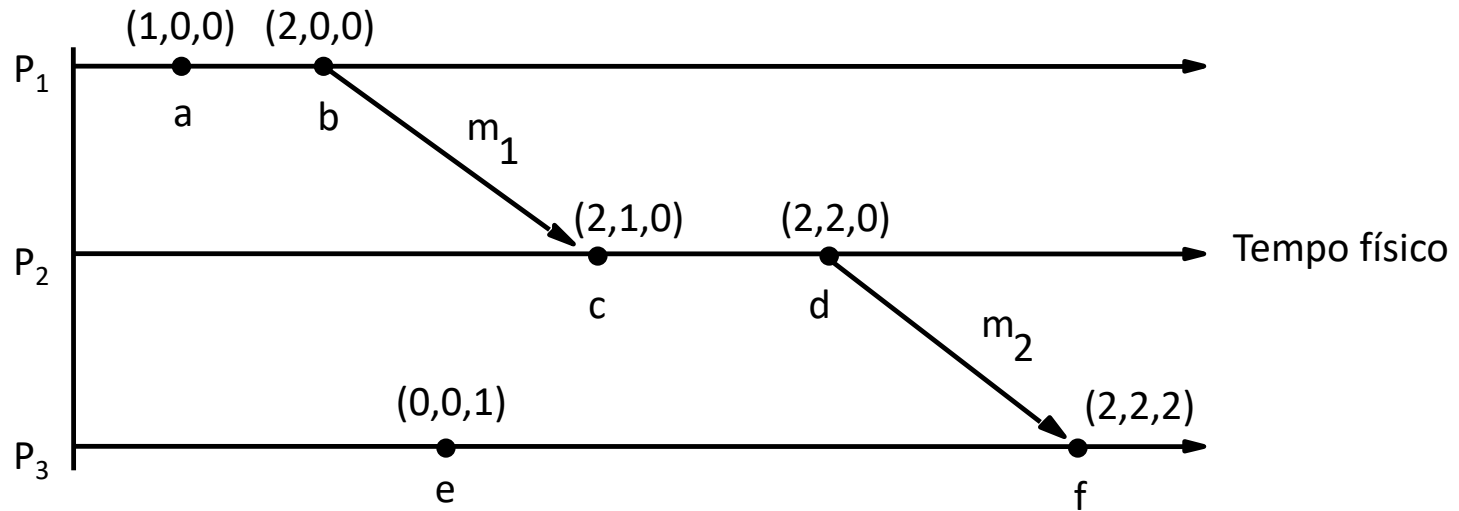


- Deficiência:
 - se $e_i \rightarrow e_j$ então $L(e_i) < L(e_j)$, **MAS o contrário não é verdadeiro**, ou seja, e_i pode não preceder e_j mesmo que $L(e_i) < L(e_j)$
 - Exemplo: $L(e) < L(b)$, mas $e \parallel b$
 - Acontece sempre quando dois processos x e y não tiverem seus relógios lógicos sincronizados
 - Falta de mensagens entre x e y
 - Mais eventos ocorrendo em um processo que em outro

Relógios vetoriais (Mattern e Fidge)

- Proposta para suprir deficiência do relógio de Lamport
 - O fato de que a partir de $L(e) < L(e')$ não concluir $e \rightarrow e'$
- Relógio vetorial para cada um dos N processos é um vetor de N inteiros
 - Cada processo P_i mantém seu relógio vetorial V_i , usado para fazer timestamp de eventos locais
- Regras para atualização de relógios:
 - **VC1**: inicialmente, $V_i[j] = 0$, para $i, j = 1, 2, \dots, N$;
 - **VC2**: imediatamente antes de P_i processar um evento, ele marca o timestamp atualizando $V_i[i] = V_i[i] + 1$
 - **VC3**: P_i inclui o valor $t = V_i$ em cada mensagem que envia
 - **VC4**: quando P_i recebe timestamp t em uma mensagem, ele atualiza $V_i[j] = \max(V_i[j], t[j])$, para $j = 1, 2, \dots, N$, e aplica VC2

Relógios vetoriais: exemplo



- Comparação de relógios vetoriais
 - $V = V'$ se e somente se $V[j] = V'[j]$, para todo $j = 1, 2, \dots, N$
 - $V \leq V'$ se e somente se $V[j] \leq V'[j]$, para todo $j = 1, 2, \dots, N$
 - $V < V'$ se e somente se $V \leq V'$ e $V \neq V'$

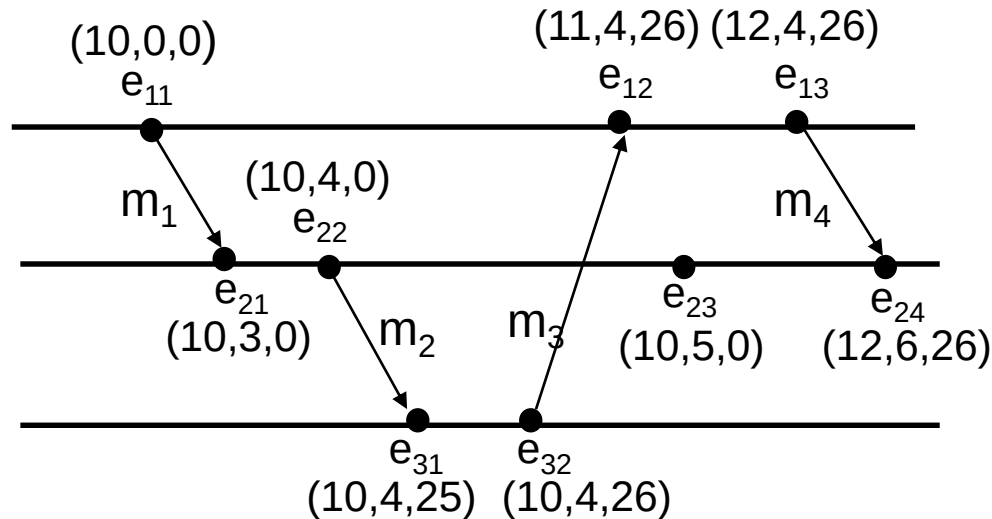
Relógios vetoriais: exemplo

Valores iniciais
(exemplo)

$L_1=(9,0,0)$ P_1

$L_2=(0,2,0)$ P_2

$L_3=(0,0,24)$ P_3



maior
 $e_{23} (10, 5, 0)$
 $e_{32} (10, 4, 26)$
 igual
 maior
 Logo, são concorrentes!

- Relógio vetorial também fornece apenas ordenamento parcial
 - e_{23} vem antes ou depois de e_{32} ?

Indo adiante em sistemas distribuídos...

- Como em um sistema centralizado, também é necessário efetuar atividades de controle
 - Exclusão mútua
 - Tratamento de deadlock
 - Detecção de término
- Realizadas por algoritmos de controle (para sistemas distribuídos)
 - Podem usar um processo coordenador ou serem totalmente distribuídos (sem coordenador)
 - Coordenador é eleito entre vários processos
 - >> **Algoritmos de eleição**

Algoritmos de eleição

- Algoritmos para escolher um processo único para desempenhar algum papel em particular (e.g., coordenador, time-server, etc)
 - Sempre que houver desconfiança da falha do coordenador
 - Necessário que todos os processos concordem na escolha
- Definições:
 - Processo inicia eleição quando ele detecta que coordenador atual falhou
 - Um processo não pode iniciar mais de uma eleição ao mesmo tempo
 - Porém, N processos poderiam iniciar N eleições ao mesmo tempo
 - Em um dado momento processo P_i pode ser participante ou não-participante
- Processo eleito deve ser único, e deve ser aquele com maior **id**
 - **id** = Qualquer valor “útil”, desde que seja único e totalmente ordenado
 - E.g., eleger processo com a menor carga: $id = \langle 1/load, i \rangle$
 - E.g., eleger processo com maior prioridade ou maior pid, etc

Algoritmos de eleição

- Algoritmos clássicos (para sistemas distribuídos):
 - Algoritmo do anel
 - Algoritmo valentão (bully)
- Em ambos
 - Cada processo P_i ($i = 1, 2..., N$) tem variável $elected_i$
 - Quando P_i se torna participante, ele seta $elected_i = \perp$
 - Ao final, deve satisfazer duas propriedades
 - Liveness
 - Algoritmo termina quando todos os processos tem $elected_i \neq \perp$, ou
 - Sofreram colapso
 - Safety
 - Ao final, todos os processos que não sofreram colapso setaram $elected_i = P$, onde P é o processo com maior id e que não tenha sofrido colapso

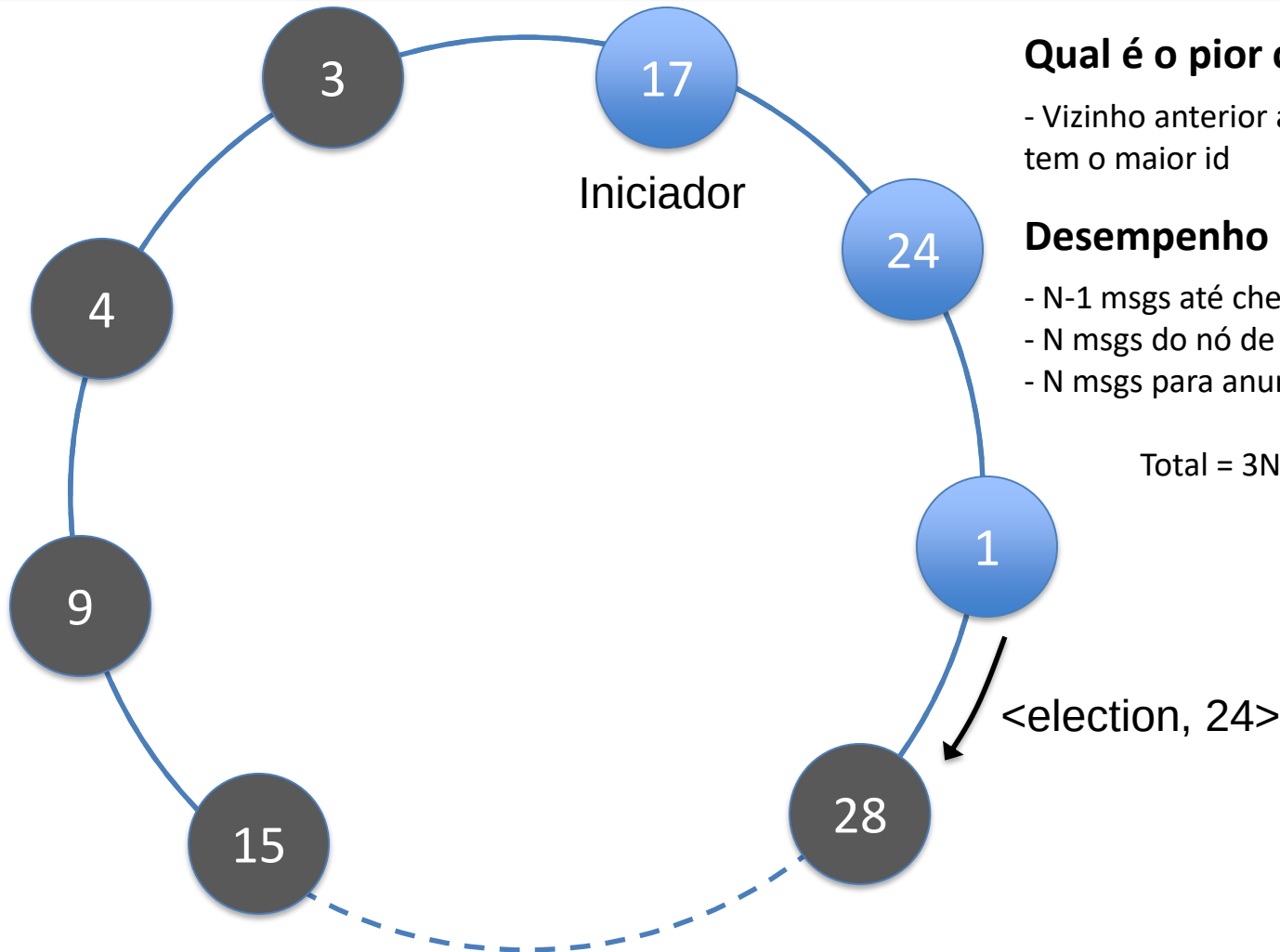
Algoritmo do anel [Chang & Roberts, 1979]

- Processo são organizados em um anel lógico (unidirecional)
 - Cada processo P_i se comunica apenas com o próximo processo $P_{(i+1) \bmod N}$
 - Mensagens enviadas no sentido horário, não tolera falhas
- Funcionamento:
 - Qualquer P_i pode iniciar eleição: marca P_i como participante, e envia mensagem m com seu id para o sucessor no anel: $m = \langle \text{election}, \text{id} \rangle$
 - Quando algum processo P_j recebe $m = \langle \text{election}, \text{id} \rangle$:
 - Compara id na mensagem com seu próprio
 - Se id na mensagem for maior, repassa a mensagem para o próximo vizinho
 - Se id na mensagem for menor e processo não-participante, substitui id e repassa
 - Porém, P_j não repassa se este já for participante
 - Se algum processo recebe na mensagem seu próprio id, então é o maior
 - Coordenador: processo se marca como não-participante, e envia mensagem $m = \langle \text{elected}, \text{id} \rangle$ para o vizinho (repassada até fazer outra volta)

P_j também se
marca como
participante



Algoritmo do anel [Chang & Roberts, 1979]



Qual é o pior caso?

- Vizinho anterior ao iniciador é aquele que tem o maior id

Desempenho no pior caso?

- $N-1$ msgs até chegar ao nó de maior id
- N msgs do nó de maior id p/ chegar nele mesmo
- N msgs para anunciar que é o coordenador

Total = $3N-1$ mensagens

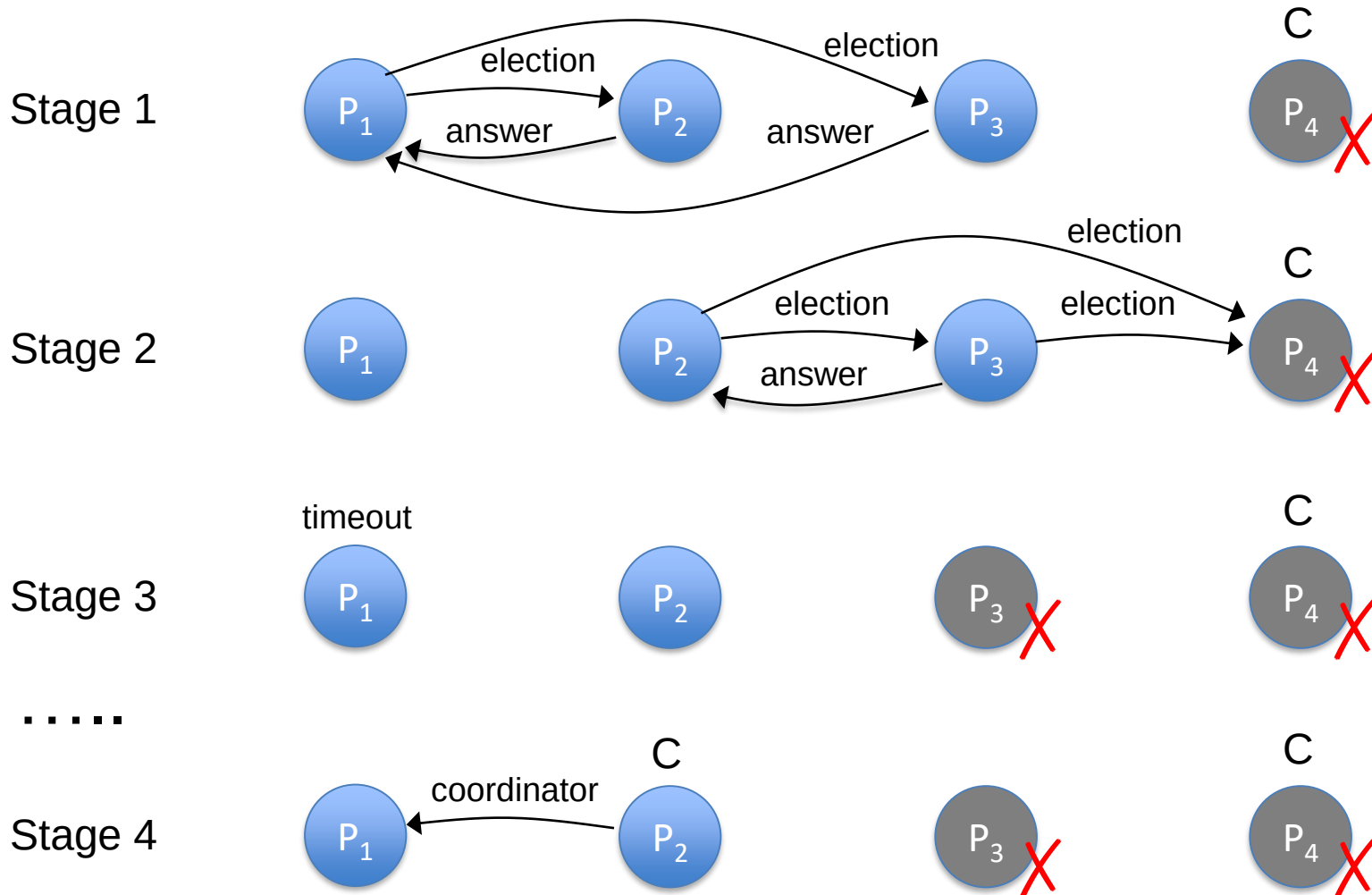
Algoritmo *bully* (valentão)

- Proposto por Garcia-Molina, 1982
- Diferenças em relação ao algoritmo do anel:
 - Permite falha de processos, mas assume entrega confiável de mensagens
 - Assume que sistema é síncrono (utiliza timeouts p/ detectar falhas)
 - Assume que cada processo sabe quais outros processos tem id maior
- Tipos de mensagens
 - `election`: usada para iniciar uma eleição
 - `answer`: usada em resposta a uma mensagem do tipo `election`
 - `coordinator`: usada para anunciar o id do processo eleito
- Eleição começa quando P_i determina que coordenador falhou
 - Síncrono: T_{trans} (max. transmissão), e $T_{process}$ (max. processamento)
 - Detector de falhas: usa timeout $T = 2T_{trans} + T_{process}$

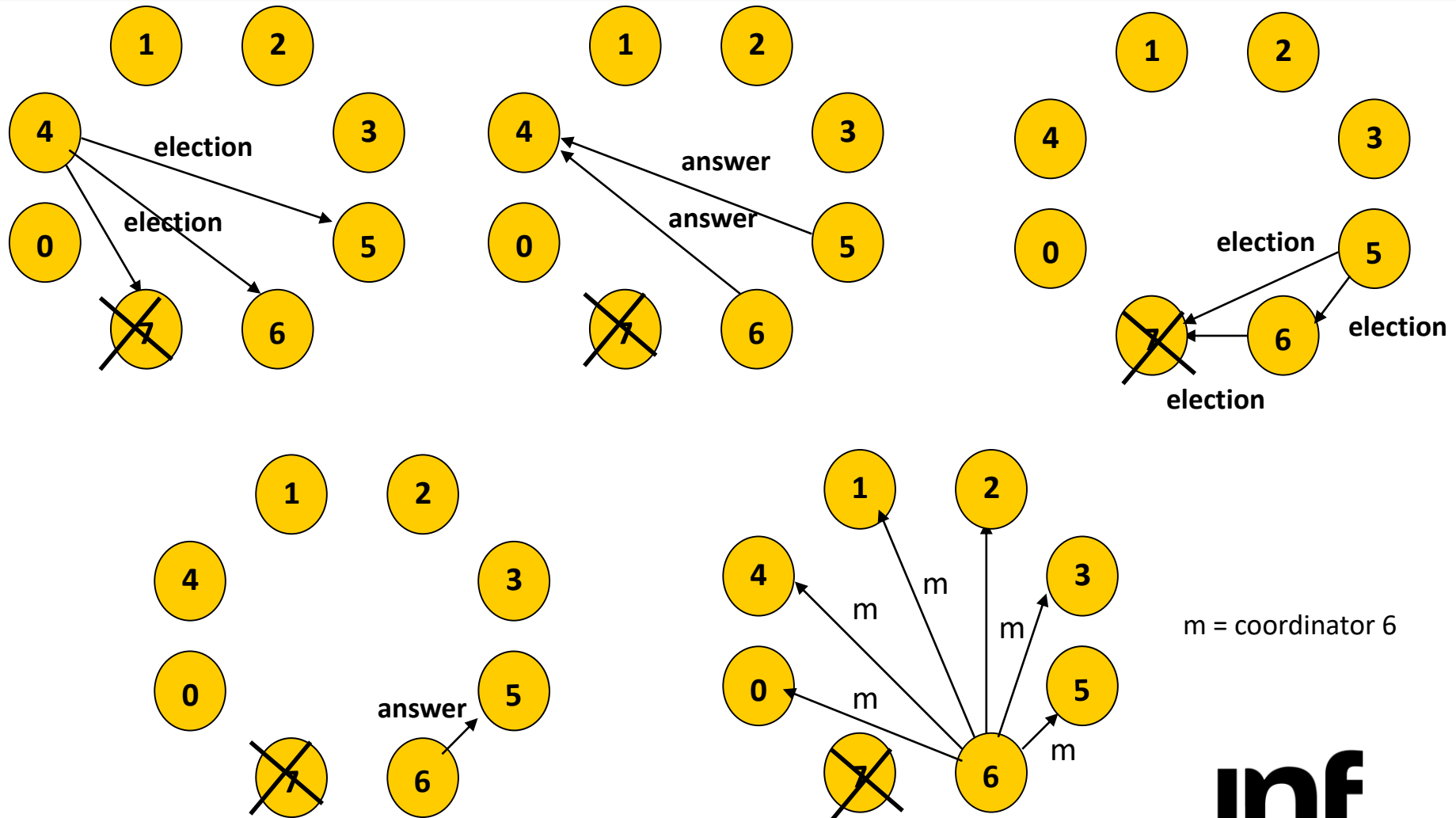
Algoritmo *bully* (valentão)

- Funcionamento:
 - P_i não obtendo resposta do coordenador envia *mensagem* `election` para todos *processos com maior id que o seu*
 - Se não receber nenhuma mensagem do tipo `answer` dentro de um *time-out* T , P_i se auto-define como novo coordenador
 - Se processo P_i recebe mensagem do tipo `answer`, isso significa que tem um processo com id superior
 - Espera por mais T' para receber mensagem `coordinator`. **E se não chegar?**
 - Processo P_j que recebe uma mensagem `election`, envia `answer` p/ emissor, e inicia sua própria eleição – a menos que já tenha feito isso!
 - Processo vencedor envia uma mensagem `coordinator` todos os outros processos (obviamente, eles terão id menor)
- Na recuperação de um processo P_k ele reinicia a eleição
 - Se possui maior id ele se tornará o coordenador e anunciará
 - Mesmo que o coordenador atual esteja ok → Bully!!!

Algoritmo *bully* (valentão)



Exemplo: Algoritmo *bully* (valentão)



Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.—
“Distributed Systems: Concepts and Design” (5th edition),
Addison Wesley, 2012
 - Capítulo 14 (seção 14.4)
 - Capítulo 15 (seção 15.3)
- Lamport, L. *“Time, clocks and the ordering of events in a distributed system”*. Communications of the ACM, Vol. 21, No. 7, pp. 558-65, 1978.